
Notifications and Negotiations — How They Work

How the notification spine and the negotiation engine actually work — the rules, the reasons the odd ones exist, and where the code is.

Aajoo Homes — Technical reference

Prepared 12 September 2026 by the Zyphex Tech development team

Verified against the code deployed to www.aajoohomes.com and build 81 of the Android app

Two systems, written up as they stand on **12 September 2026**, after the rebuild of that day. Everything below is the behaviour of the code now running on www.aajoohomes.com and in build 81 of the Android app — not a design intent.

It is written for whoever has to reason about these systems next: which function to change, which one not to, and why a given rule exists. Where a rule looks odd, the reason it looks odd is given, because those are the ones somebody well-meaning removes.

Part One — Notifications

1. Two stores, and why it matters

There are **two** in-app notification tables, with different readers. Sending to the wrong one means the notification is written, and nobody ever sees it.

Table	Whose notifications	Read by
<code>tbl_user_notifications</code>	Guest	the guest bell, <code>/account/notifications</code> , the app's renter notifications
<code>tbl_notifications</code>	Host / Admin	the host portal, <code>/host/notifications</code> , the admin queue

`tbl_notifications.notify()` **refuses a GUEST role outright** and logs why. A GUEST row in the host/admin table is read by nobody — that is how a check-in notice was lost once, and the guard exists so it cannot happen again.

2. Two choke points

Every notification on the platform passes through one of exactly two functions:

Function	Recipients
<code>utils/methods.sendNotification(userId, title, message, propId, payload, bookingId, bookPriId)</code>	Guests
<code>models/tbl_notifications.notify({ role, recipientId, title, body, linkPath, category })</code>	Hosts and admins

Everything in the rest of this section is attached **there**, not at the twenty-odd call sites. That is the whole design decision: a call site that forgets its category, its email or its socket emit is a silent gap, and the next call site added would forget too.

3. What happens, in order

Guest path — `sendNotification`

1. **The row is written.** Always, and first. This is the ledger: the record of what the platform told you and when. No preference can suppress it.
2. `notification:new` **is emitted** on the socket to the room `user_<id>`. This is what makes a popup appear on whatever screen the person is looking at, with no reload and no navigation.
3. **An email is queued** — `notificationEmail.mailNotification()`. Not awaited, and it never rejects: a notification must not wait on a mail transport, and a mail failure must cost the mail and nothing else.
4. **FCM push** to *every* device this person has registered (`tbl_notify_device`), not only the most recent. Tokens FCM reports as dead are pruned on the response, so an uninstalled app is not retried for ever.

Host path — `notify`

Row → push → socket → email. ADMIN is skipped throughout: admins are not users. They register no devices, have no personal socket room, and have no address on `tbl_user_creds`. A broadcast row (`recipientId` null) has nobody to reach either.

One detail worth knowing

`un_bookingId` is the sixth positional argument and almost nobody passed it — 236 of 367 live rows had none, including every *"Your booking is confirmed"*. That column is what both clients read to open **that** stay rather than a list, so without it the notification could only ever open Upcoming Stays, and on an account whose stay had since finished, an **empty** Upcoming Stays. It is now taken from the payload when the argument is absent.

`tbl_users` **has no email column.** Every address lives on `tbl_user_creds.cred_user_email`, filtered by `cred_user_isDelete = 0`. `utils/userContact.js` is the only reader, with a 60-second cache.

This matters because the old host-offer email selected `user_email` on `tbl_users` — a column that does not exist — and therefore **threw on every single offer for as long as the feature had existed**, caught and logged as a warning. No host was ever emailed about an offer until 12 September.

4. Categories are derived, never passed

`utils/notificationCategory.js` decides what kind of thing a notification is, by reading the **wording first** and the stored `payload.type` only as a fallback.

Five categories, chosen to be what a person would recognise on a settings screen rather than what the database calls things:

`booking` · `payment` · `offer` · `support` · `promotions`

Order of matching is deliberate. *"Offer accepted — book within 24 hours"* is an **offer**, not a booking, even though the word "book" appears in it. *"Booking cancelled"* is a booking matter and still a cancellation.

Why the wording and not the stored type: the stored types are demonstrably wrong. There is a live row titled *"Your Property has been Booked"*, body *"Booking Successfull"*, carrying the type `negotiation_request`. The title is written for a human to read and describes the event; the type was whatever the call site happened to pass.

An unmatched notification falls back to `booking`, never `promotions`. An unrecognised notification is far more likely to be something the person needs than something they can miss, and the cost of the two mistakes is not symmetric: a stray notification is an annoyance, a suppressed cancellation is a guest turning up at a house that is not expecting them.

5. Preferences

`utils/notificationPrefs.js` is the only thing that reads `tbl_notification_prefs`. Everything else calls `allows(userId, channel, notification)` and gets a plain boolean.

Channel	Switchable	Why
Push	Per category	The interrupting channel. The one people actually want control of
Notification email	Per category	A copy of the in-app notification, so it reaches someone with the app closed. Same interruption in a different place, so the same preference shape
WhatsApp	Promotions only	The platform's one marketing channel, and marketing is what a person is entitled to stop
In-app	No	It is the record. A cancellation never written because a box was unticked is a support case nobody can answer
Transactional email	No	Receipts, confirmations, password resets. Offering to turn those off is offering to lose your own records — and they do not pass through here at all

Defaults are **everything on**. The absence of a row means exactly that. The notification-email default is on for every category deliberately: the whole point of adding it was that notifications were sitting silently in a bell nobody opened.

Every failed read answers `true`

A database hiccup must not silently mute a person's notifications. The failure would be invisible both to them and to us, and the thing not delivered might be the one that mattered.

One switch, two channels

A category toggle now writes **both** the `push` and the `email` map. Until 12 September it wrote `push` alone, while the email copy was gated by the same category on a map nothing on either client could reach — so a guest who turned "Offers and negotiations" off went on receiving an email for every counter, with no way to say otherwise.

A second column of switches would be the other answer, and it is the wrong one: nobody wants a counter-offer by mail but not by push, and a grid of twelve toggles is how a preferences screen stops being read. The **category** is the question; the channels are how we ask it.

6. What is never emailed, whatever the preference says

`notificationCategory.emailable()` excludes exactly two kinds:

Excluded	Why
Chat messages	A conversation produces a notification per line, and a mailbox is not a chat window. The in-app notification and the push already reach them, and the inbox is one tap away
Payment received	The confirmation a person gets when their own payment goes through. They are looking at the screen that just took the money, and a separate receipt and invoice already follow. Two mails about one payment is how a receipt gets ignored

Everything else is emailed: bookings, cancellations, check-ins, offers and counters, refunds, payouts, support replies, KYC, listing decisions.

Refunds, payouts and cancellations are matched FIRST and always return true, even though all three read as payment wording. Each is money the person is waiting on and will chase support about if they do not hear — the exact opposite of noise.

The payment exclusion is deliberately narrow: it catches the **success** confirmation only. A payment that failed, is pending, is due or was declined is something a person very much needs to be told about, and is still emailed.

7. Where a tap goes — four readers, one rule

Four separate things resolve the destination of one notification:

- `utils/notificationRoute.js` — the server, for the link in the email
- `notificationLink.ts` — the website's bell, list and live popup
- `notification_link.dart` — the app
- the live-popup handler, which asks the same function the bell does

They read the same evidence in the same order, because **a mail that opens Bookings beside a bell entry that opens Negotiations, for the same row, is a person who cannot work out what happened.**

Offers route to `/account/negotiations` for guests and `/host/negotiations` for hosts. The web resolver used to send a guest's offer notification to the dashboard, on the reasoning that guests had no negotiations page — they have had one since 23 August, and sending them anywhere else was the dead end the client reported: an offer answered by an automatic counter, and no way to reach the counter.

Routes are validated against a known set, so a stale or invented payload route is rejected rather than followed to a 404.

8. The live channel

Side	What it is
Server	<code>utils/liveChannel.js</code> — <code>emitToUser</code> , <code>emitToUsers</code> , <code>roomFor</code> , <code>isLive</code> . It never throws; a socket failure costs the popup and nothing else. <code>sockets/index.js</code> joins <code>user_<id></code> from the authenticated handshake
Web	<code>liveNotifications.ts</code> , a module singleton — one socket per person
App	FCM in the background, the same socket in the foreground

The web singleton is not a style preference. Four things want the same stream at once — the app-wide toast, the header bell's count, the guest and host negotiation panels, the notifications page. A hook per consumer is a socket per consumer: four connections for one person, four handshakes, against a server capped at a thousand.

The socket is an addition, never the only route. If it never connects — blocked, proxied, a sleeping laptop — every screen still works exactly as it did: the bell still polls, the lists still fetch. Nothing here is the sole way any fact arrives.

The popup itself lasts 8 seconds, sits bottom-right with a 76px bottom margin to clear the support chat bubble, and suppresses itself when the person is already looking at the page it would send them to.

Part Two — Negotiations

9. The three prices

Every listing carries three. The host sets them once, in Step 4 of the listing wizard.

Price	Who sees it	What it means
List (<code>property_price</code>)	The guest — it is the price on the page	What the stay costs without negotiating
Ideal (<code>property_ideal_price</code>)	Nobody outside the platform	The host's target. An offer at or above it is accepted with no host involvement
Minimum (<code>property_mini_price</code>)	Nobody. Ever.	The host's floor. Only the host may sell below their target, and only by saying so themselves

They are the prices for the nights being argued over

This is the change that touches every figure in the system. A listing's nightly column is what it costs on an ordinary night; weekend and seasonal rates make a particular night cost something else. **Every number the engine quotes is now the number for the dates in the offer** — the price the guest is shown, the ceiling an offer is measured against, the accept line, the automatic answer, the discount the coupon carries, and both sides' negotiation cards.

`negotiationService.tiersForDates(property, from, to)` is the single way to ask. It falls back to the flat columns loudly if the rates engine cannot answer, so a quote hiccup degrades the number and never the feature.

The reason this is stated so firmly is in Appendix A: **eight** separate surfaces were reading a dated price out of an undated column, and one of them cost the host money.

10. Where it is stored

Table	Holds
<code>tbl_negotiation_offers</code>	Every message in every thread — price, sender, status, <code>offer_number</code> , the stay dates
<code>tbl_negotiation_log</code>	The ledger: who did what and when, with the floor and ceiling snapshotted on each row
<code>tbl_nagotiate_messages</code>	The older socket-chat system
<code>property_negotiation</code>	<code>pn_enabled</code> , <code>pn_minimum_price</code> , <code>pn_expiry_hours</code>
<code>tbl_coupons</code>	Where a struck deal actually lives

There are two chat systems on this platform. Negotiations live in `tbl_nagotiate_messages`; booking conversations live in `tbl_messages`, the regular inbox, as the website has always done. Assuming there was only one is how a bug once got fixed on the wrong screen.

`tbl_negotiation_offers` has `created_at` and **no** `updated_at` — the model sets `updatedAt: false`. A declined row therefore cannot say *when* it was declined, which is why the same-day lockout reads the ledger instead.

11. The decision

`services/negotiationEngine.js` — `decideOffer({ offer, min, ideal, max, round })`.

Condition	Action	What the guest sees
No floor set, or a nonsense offer	<code>unavailable</code>	This stay is not open to offers
Above the dated list price	<code>reject</code>	"These dates are ₹X a night — offer less than that, or just book it"
At or above <code>max(ideal, min)</code>	<code>accept</code>	Accepted at the price they named — not the ideal, not the list
Below it, round 1	<code>auto_counter</code>	An answer in about a second, in the host's name
Below it, round 2 or later	<code>escalate_to_host</code>	It goes to a person

The accept line is the **ideal**, clamped up to the floor so a listing whose ideal was somehow set below its own minimum still never sells under it. The client moved that line from the minimum to the ideal and confirmed it again on 9 September.

An offer **below the floor** is countered exactly like one merely below the ideal. The floor is an internal number the guest was never shown, and answering it differently would be the probe that finds it.

12. What the automatic answer costs

```
list      = max(accept, listPrice)
ceiling   = max(accept, list - 50)          COUNTER_STEP = 50
closeness = clamp(offer ÷ accept, 0, 1)
price     = floor( (ceiling - (ceiling - accept) × closeness) ÷ 50 ) × 50
```

The worse the offer, the firmer the answer. On a ₹2,500 list with a ₹2,300 accept line: an offer of ₹1,800 is answered ₹2,300; an offer of ₹100 would be answered ₹2,400.

Two consequences worth stating to anyone who might "simplify" this:

1. **No automatic answer is ever worth less to the host than an instant acceptance would have been.** Every one sits at or above the accept line.
2. **A token discount is always given** — at worst `COUNTER_STEP` below the list price. Quoting the sticker price back at somebody is not a counter, and a feature called "negotiate your stay" that answers with the price already on the screen has not negotiated anything.

The floor is not in this arithmetic at all

Not clamped against, not guarded by a share of it, not mentioned. The answer is built from the accept line and the list price, and can never land below the accept line, which is itself never below the floor.

So **there is no offer whose answer moves when the minimum moves**, and therefore nothing a guest can do to find it. This is stronger than the version it replaced, which needed a guard at 90% of the line purely to stay clear of the floor.

The one number a determined guest can work backwards to is the **ideal**, and that is unavoidable in any rule that guarantees a counter at or above it. It costs nothing: knowing the ideal only tells them they could have offered it and been accepted at it — which is **more** than the counter in front of them, never less.

It was demonstrated, not asserted

Three offers a rupee apart, straddling the floor — one below, one exactly on it, one above — each sent as the opening offer of its own negotiation. The three answers came back as **byte-for-byte identical PNG files**. Section 6 of the user-journey document carries the photographs.

The removed settings

Step 4 of the wizard also collected `pn_auto_accept_above`, `pn_auto_reject_below` and `pn_max_negotiation_percent`, and the engine briefly read all three. It no longer does, and the controls are gone from the form: a reject line is only useful to a guest **if the refusal quotes it**, and quoting it hands out a private number the spec forbids twice over.

The host keeps the same power without the leak. Every counter-back reaches them, and they decline it themselves.

13. Rounds, and how long things live

There is no limit on rounds. The three-offer cap was removed on the client's instruction: it ended arguments hosts were still willing to have. A thread runs until it reaches a terminal status — `accepted`, `declined` or `expired` — at which point `splitSessions` ends the session.

Clock	Length	Set by
An offer waiting on a host	30 minutes default	<code>NEGOTIATION_EXPIRY_MINUTES</code> , or the host's <code>pn_expiry_hours</code> if longer
An accepted deal	Until midnight IST	—
The price a decline leaves behind	60 minutes	<code>PARTING_OFFER_MINUTES</code>
Before the guest is told the host is away	90 seconds	—

A host's configured window only ever **lengthens** the allowance. Before this was read, a host who asked for a day to think got thirty minutes.

14. How a deal reaches checkout

Every struck deal becomes a **personal, date-locked percentage coupon**, because that is the only instrument the booking flow has.

Property	Value
Code	<code>DEAL{property}{guest}{ref}</code> , sliced to 20 characters
Usage limit	1
Locked to	The exact nights, and the party size
Valid until	Midnight IST (accepted) or 60 minutes (parting)
Percentage	Worked out against the dated rate

The 20-character slice is the column width, and it has bitten before: a code built from `auto_${Date.now()}` truncated to the same 20 characters for six different offers, so one coupon was silently refreshed by all six and was dead for every one of them.

Because the coupon stores a **percentage**, turning it back into a per-night figure requires the same dated base it was minted against. Getting that wrong is how the parting-price sentence came to read ₹1,840 on a rail that was charging ₹2,300.

15. The decline

When a host declines, three things happen.

1. **The thread ends.**
2. **Their last counter stays on the table for one hour**, minted as a coupon titled "Host's last price", locked to those nights, single use, and dying with the hour whether or not it is used. It is the host's own number, never a figure the platform invented.
3. **That guest cannot open another negotiation on that listing until midnight IST.**

Point 3 is not decoration. Without it, removing the round cap left a door wide open: a decline is a terminal status, so it

ends the session

— and the very next offer would start a fresh one with a clean round count. A guest could be declined and re-open the same argument a second later, for ever. The cap used to hide that; it does not any more.

The day is an **IST day, not a rolling twenty-four hours**. Declined at 9am and you are free again at midnight; declined at 11pm and you are free in an hour. That is a rule a person can hold in their head, and it is the same unit the negotiated-price deadline uses.

It is read from the ledger rather than the offers table, because that table cannot say when a row was declined, and it **fails open**: a database hiccup must not silently bar a guest and tell them they were declined today when they were not.

16. The lock, on both clients

`declineLockFor()` returns `{ locked, reason, until, price, code, from, to }`. The reason is one of three, and each needs different words.

Reason	The button says	Scope
<code>accepted</code>	"Already agreed for these dates"	Those nights only
<code>parting</code>	"This negotiation has ended"	Those nights only
<code>declined</code>	"Available again tomorrow"	The whole listing, until midnight IST

The first two are **date-scoped**: a guest who agreed a price for this weekend may still negotiate next weekend on the same property. `lockFrom` / `lockTo` travel on the payload so both clients answer "does this bar the dates on screen?" from one call and cannot drift over what "these dates" means.

The button is greyed with the reason, never hidden. A control that vanishes reads as a broken feature, and two of these three come back on their own. Before this, the button stayed live and the server refused the filled-in form afterwards — the dead end the client asked us to remove.

17. What a message is called

A **status** describes what happened **TO** a message, not what its sender did. Whoever acts on an offer is always the other side from whoever sent it.

Read the other way round, a renter whose offer the host had declined was shown **"You declined ₹650"** on their own screen — at the end of the one thread that needed to explain itself.

Viewer	Their own message	The other side's
Guest	You offered · Your offer was accepted / declined / expired	Host countered · Accepted · Declined · Expired
Host	You countered · Your counter was accepted / declined	Guest offered · Accepted · Declined
Host, written by the platform	Answered for you, at your price · Accepted for you	—

That last row is the point of the whole table. The platform answers round one in the host's name, and **a host must be able to tell which words in their own conversation are theirs.**

One shared rule serves four screens — the website's guest and host negotiation pages, the app's guest list and the app's host list. They had four copies of one paragraph and drifted for a fortnight over whose move it was.

18. Live events

`negotiation:new_offer` · `negotiation:auto_countered` · `negotiation:auto_booked` · `negotiation:host_reply` · `negotiation:guest_reply`

All are emitted to **both** sides, so a panel can repaint itself rather than reloading the dashboard — the client's request that negotiations feel "live like WhatsApp: it should not reload the whole dashboard, only the window where they are visible should be updated."

19. The rules around the edges

- **Negotiation is for stays starting today.** Tomorrow or later is an advance booking: the listed price applies, reservable with 10% now and the balance before check-in. The rule lives on the dates in `submitOffer`, the one door the website, the app and the bot all come through. It used to live on the clients, as a marker on the pre-booking journey's links — so the same future stay opened from search negotiated freely, and 17 of the 23 deals ever struck were for future dates.
- `pn_enabled` **rides on the property payload**, so a guest discovers that a host takes fixed prices only *before* filling in a form that will be refused.
- **An offer above what the stay costs is a correction, not a negotiation.** It is refused with that figure named, and the guest is pointed at Book Now.

- **Nothing that differs with the price a guest names may be exposed.** `belowMinimum` is host-payload only; the guest's payload carries the listed price and nothing else from the tier object. The host's own card shows *whether* an offer fell below their minimum and never the figure — that only adds something to leak in a screenshot.

Appendix A — One fault, eight places

On 12 September, while photographing the user-journey document and driving the cross-device suite, the same mistake was found in eight separate places: a price that depends on the dates, read from a column that does not.

#	Where	What it did
1	<code>submitOffer</code> 's ceiling	Refused an offer that was under what the weekend actually cost
2	The web rail's struck-through headline	Printed ₹2,000 beside a negotiated ₹2,400 — a discount reading as a price rise
3	<code>declineLockFor</code> 's parting price	Quoted 8% off ₹2,000 as ₹1,840, on a rail charging ₹2,300
4	The host's negotiation card	"Guest offered ₹2,400 / Your price ₹2,000" — an offer ₹100 under the ask, shown as ₹400 over
5	The guest's negotiation card	"Listed at ₹2,000/night", directly above the stay dates
6	"for your dates · listed at ₹2,000/night"	Named the one price that was not the list price for those dates
7	The app's offer sheet	"Listed at ₹900 / night" and a client-side refusal, on nights costing ₹1,200
8	The counter-back verdict	Accepted ₹850 on a stay whose accept line was ₹1,050 — the host sold a night ₹200 under what they would have agreed to, and was never asked

Only the last one cost money. It was found by driving the Android suite: the guest opened at ₹900, was correctly answered ₹1,050 from the dated tiers, and then countered back at ₹850 — which clears the **flat** ideal of ₹850 and was taken on the spot. Every other number in that thread was already dated, so the two halves of one negotiation were working from two different sets of numbers.

The underlying cause was that the derivation lived inline inside `submitOffer`, where only `submitOffer` could reach it. It is now `tiersForDates()`, and everything asks it.

Appendix B — What changed on 12 September

	Before	Now
Rounds	Three offers per guest per stay, then the thread closed	No limit. A thread runs until it is settled
An automatic acceptance	Wrote the guest's message and no answer, so accepted offers read as a column of unanswered messages	Writes both sides, labelled so the host knows which words are theirs
Message labels	By direction alone — "You declined ₹650" on the screen of the person who had been declined	By what happened to the message. One rule, four screens
A decline	Thread closed, guest left with nothing	The host's last price for one hour, locked to those nights, and the day closed to further offers
A held deal	The offer button stayed live and the server refused the form afterwards	Greyed, with the reason and what to do instead
Prices	The flat nightly column, while the offer was judged against the dates	Everything is the figure for the nights being argued over
Notifications	Sat silently in the bell, which only moved on a 60-second poll	A popup on any screen, an email, and a push — the instant it happens
Emails about offers	Selected a column that does not exist; threw on every offer, in silence	Sent, and read at the address the account actually uses
The acceptance wording	"That is at or above what the host will take"	Says the host agreed and the booking can go ahead — nothing about where the line sits
Notification preferences	One switch wrote <code>push</code> only; email went out regardless	One switch, both interrupting channels

Appendix C — Where the code is

Concern	File
Guest notification choke point	<code>utils/methods.js</code> → <code>sendNotification</code> , <code>pushToUser</code>
Host/admin choke point	<code>models/tbl_notifications.js</code> → <code>notify</code>
Category rules	<code>utils/notificationCategory.js</code>
Preferences	<code>utils/notificationPrefs.js</code>
Notification email	<code>utils/notificationEmail.js</code>
Email destination	<code>utils/notificationRoute.js</code>
Recipient address	<code>utils/userContact.js</code>
Socket transport	<code>utils/liveChannel.js</code> , <code>sockets/index.js</code>
Web socket singleton	<code>src/redesign/lib/liveNotifications.ts</code>

Concern	File
Web popup	<code>src/redesign/components/LiveNotifications.tsx</code>
The decision engine	<code>services/negotiationEngine.js</code>
Tiers, offers, locks	<code>services/negotiationService.js</code>
Offer expiry sweep	<code>services/negotiationExpiry.js</code>
Coupon minting	<code>utils/negotiationCoupon.js</code>
Host responses	<code>controllers/host.controller.js</code>
Guest responses	<code>controllers/user.controller.js</code>
Message labels (web)	<code>src/redesign/lib/negotiationLabels.ts</code>
Message labels (app)	<code>lib/utils/transcript_label.dart</code>
Offer ceiling (web / app)	<code>src/redesign/lib/offerCeiling.ts</code> · <code>lib/utils/offer_ceiling.dart</code>
The lock (app)	<code>lib/models/single_property_response.dart</code> → <code>NegotiationLock</code>

Prepared 12 September 2026 by the ZypheX Tech development team. Every rule stated here is the behaviour of the code deployed to www.aajoohomes.com and build 81 of the Android app on that date, verified by driving the live site and a real device rather than by reading the source alone.